

GAME DESIGN & DEVELOPMENT

2D Shooter · Unity Engine

Master-Level Exam Notes | Weeks 1 – 8
Bachelor's Level · Exam-Focused · 2025 / 2026

Includes: Core Concepts · Code Examples · Comparison Tables · Interview Q&A

Table of Contents

1. Introduction to Game Programming

- What is Game Programming?
- How Games are Made
- The Game Loop

2. Game Design & The Development Team

- The Game Designer's Role
- Development Team Roles
- Costs & Limitations

3. Game Engines & Unity Fundamentals

- What is a Game Engine?
- Unity Editor Overview
- Project Setup & Asset Import
- Solar System Simulation Project

4. Level Design & Game Systems

- Principles of Level Design
- Game System Architecture
- Screenshots & Documentation

5. Player Setup — Visual, Controls & Health

- New Unity Input System
- Player Visual Setup
- Health & Damage System
- Combat Mechanics

6. Cameras, Level Environment & Game Loop

- Camera Framing Techniques
- Environment & Sprite Setup
- UI Management & Game Loop
- Audio & Music Integration

7. Unity Programming & C# Fundamentals

- Object-Oriented Thinking in Unity
- Anatomy of a C# Script
- Variables, Conditionals, Loops
- Functions & Classes

8. Tilemaps, Animation & Advanced Setup

- Tile Palette & Rule Tiles
- Animated Tiles
- Player Physics & Input Refinement
- Canvases & UI

9. Interview & Exam Questions

10. Quick-Revision Summary

WEEK 1 — Introduction to Game Programming

1.1 What is Game Programming?

Game programming is the software development process dedicated to the creation of video games. It is a multidisciplinary field that draws on computer science, mathematics, physics simulation, artificial intelligence, and creative design. Unlike general software engineering, game programming must operate under strict real-time constraints — typically delivering 30 to 60 rendered frames every second — while simultaneously processing player input, running physics, executing AI logic, and managing audio.

Key Insight: Game programming is NOT just writing code. It is the art of making an interactive experience feel alive, responsive, and fun — within hard performance budgets.

Core Sub-disciplines

Discipline	Responsibility	Example
Engine Programming	Core systems: rendering, physics, input	Unity / Unreal source code
Gameplay Programming	Player mechanics, enemies, items	Player health script
Graphics Programming	Shaders, lighting, visual effects	Shader Graph in Unity
AI Programming	Enemy behaviour, pathfinding	NavMesh Agent
Tools Programming	Level editors, pipeline tools	Custom Unity Editor windows
Network Programming	Multiplayer sync, latency handling	Unity Netcode

1.2 The Game Process — How Games are Made

Professional game development follows a structured pipeline composed of five overlapping phases. Understanding these phases is critical for both exam answers and industry readiness.

Phase: Pre-Production

Concept, GDD (Game Design Document), prototyping, feasibility. The most important phase — mistakes here cost the most to fix later.

Phase: Production

Full team actively builds assets, code, levels, and audio. Sprints / milestones used to track progress.

Phase: Alpha

All features implemented but buggy. Internal testing begins. Feature-lock enforced.

Phase: Beta

Content-complete. External / public testing. Bug fixing only.

Phase: Gold / Release

Final build submitted to platform holders (Steam, console cert, etc.) or shipped directly.

■ EXAM NOTE: Exams frequently ask you to ORDER these phases correctly or describe what distinguishes Alpha from Beta. Alpha = feature-complete; Beta = content-complete.

1.3 The Game Loop

The game loop is the central heartbeat of every real-time game. It continuously executes three operations: process input, update game state, and render the frame.

Pseudo-code: Basic Game Loop

```
// Called once at startup
void Start() {
    InitialiseGame();          // load assets, set initial state
}

// Called every frame by the engine
void Update() {
    ProcessInput();            // read keyboard / mouse / gamepad
    UpdatePhysics(deltaTime);  // move objects, detect collisions
    UpdateGameLogic();         // AI, scoring, win/lose checks
    Render();                  // draw the current frame
}
```

In Unity, Start() and Update() are lifecycle methods provided by MonoBehaviour. The engine calls Update() once per frame. deltaTime is the elapsed time since the last frame and is used to make movement frame-rate independent.

■ TIP: Always multiply movement values by Time.deltaTime in Unity to ensure consistent speed regardless of frame rate.

Loop Component	Unity Method	Purpose
Input Processing	Update()	Read Input.GetAxis(), new Input System
Physics Update	FixedUpdate()	Apply forces, detect collisions (fixed timestep)
Late Processing	LateUpdate()	Camera follow after all movement is done
Rendering	Handled by engine	Culling, draw calls, GPU submission

WEEK 2 — Game Design & The Development Team

2.1 The Game Designer's Role

A Game Designer is responsible for crafting the rules, mechanics, narrative, and player experience of a game. They bridge creative vision and technical implementation. The designer does NOT necessarily write code, but must deeply understand what is technically feasible.

Core Responsibilities

- Write and maintain the Game Design Document (GDD) — the single source of truth for the project.
- Define gameplay mechanics, win/lose conditions, and player progression systems.
- Design levels, enemy placement, pacing, and difficulty curves.
- Collaborate with programmers, artists, and audio engineers to realise the vision.
- Iterate based on playtesting feedback — game design is never done on the first attempt.

2.2 The Game Development Team

Role	Primary Responsibility	Key Deliverable
Game Designer	Mechanics, rules, level layout	GDD, level files
Programmer	Code systems, tools, engine integration	Source code, scripts
Artist / Animator	2D/3D assets, textures, animations	Sprite sheets, models
Audio Engineer	Sound effects, music, mixing	WAV/OGG audio files
Producer	Schedule, budget, milestone tracking	Gantt chart, build reports
QA Tester	Bug finding, regression testing	Bug reports
Technical Artist	Bridge between art and code	Shaders, VFX, pipelines

2.3 Costs & Limitations

Game development is constrained by three fundamental resources: time, money, and scope. The Project Management Triangle (also called the Iron Triangle) illustrates that you can only optimise two of three: Fast, Cheap, or Good.

Common Budget Categories

Category	% of Budget (Typical)	Notes
Salaries	60–70%	Largest cost — skilled developers are expensive
Tools/Licences	5–10%	Engine fees, middleware, asset store
Marketing	10–20%	Trailers, influencer campaigns, events
QA & Testing	5–10%	Regression testing, certification (consoles)
Miscellaneous	5%	Office, travel, hardware

Limitations Programmers Must Respect

- Performance budget: target platform CPU/GPU determines polygon count, draw calls, and script complexity.
- Memory budget: consoles and mobile devices have fixed RAM — assets must be sized accordingly.
- Storage budget: download size affects user acquisition (especially mobile).
- Schedule constraints: feature creep is the #1 killer of indie projects.

WEEK 3–4 — Game Engines & Unity Fundamentals

3.1 What is a Game Engine?

A game engine is a software framework that provides reusable systems for building games. Instead of writing a physics engine, renderer, or audio manager from scratch, developers use the engine's pre-built systems and focus on gameplay logic.

System	What the Engine Provides	Unity Feature
Rendering	2D/3D drawing, lighting, shaders	Universal Render Pipeline (URP)
Physics	Rigidbody, collision, joints	PhysX (3D), Box2D (2D)
Input	Keyboard, mouse, gamepad abstraction	New Input System package
Audio	Spatial sound, mixing, compression	AudioSource / AudioMixer
Animation	Keyframes, state machines, blend trees	Animator + Animation clips
Scripting	Game logic in C# or visual scripting	MonoBehaviour C# scripts
Editor	Scene editing, asset management	Unity Editor (Inspector, Scene view)

Popular Game Engines Comparison

Engine	Language	Best For	Licence
Unity	C#	2D, mobile, indie, XR	Free / Pro (revenue-based)
Unreal Engine	C++ / BP	AAA 3D, cinematic quality	5% royalty over \$1M
Godot	GDScript/C#	Open-source indie	MIT (free forever)
GameMaker	GML	2D games, beginners	Subscription

3.2 Unity Editor Overview

The Unity Editor is an integrated development environment (IDE) for creating games. It combines a scene viewer, asset database, inspector, and build system into one application.

Key Editor Windows

Scene View

3D/2D viewport for arranging GameObjects. Use WASD + right-click to fly.

Game View

Preview of what the camera renders. Press Play to run the game.

Hierarchy

Tree view of all GameObjects in the current Scene. Parent-child relationships shown here.

Inspector

Component properties of the selected GameObject. Drag-and-drop fields.

Project

Asset file browser. Assets imported here are usable in the Scene.

Console

Debug.Log() messages, warnings (yellow), and errors (red) appear here.

Package Manager

Install official and community packages (e.g., Input System, Cinemachine).

■ **EXAM NOTE: Memorise all 7 editor windows and their functions — this is a very common short-answer exam question.**

3.3 Unity Project Setup & Asset Import

Creating a Project

- Open Unity Hub → New Project.
- Choose template: 2D (for this course's shooter project).
- Name the project and choose a local folder.
- Click Create — Unity generates the default folder structure.

Default Folder Structure

Unity Project Folders

Assets/	← YOUR working folder (scripts, sprites, scenes, audio)
■ Scenes/	← .unity scene files
■ Scripts/	← C# MonoBehaviour scripts
■ Sprites/	← 2D image assets (.png, .jpg)
■ Audio/	← .wav / .ogg / .mp3 files
■ Prefabs/	← reusable pre-configured GameObjects
■ Materials/	← Material / shader assets
Packages/	← Package Manager manifests (do NOT edit manually)
ProjectSettings/	← Build, physics, input settings
Library/	← Engine cache – NEVER commit to version control

Importing Assets

- Drag files into the Project window, OR right-click → Import New Asset.
- Unity automatically creates import settings (.meta files) — always commit .meta files to Git.
- For sprites: set Texture Type = Sprite (2D and UI) in Inspector.
- For audio: set Compression Format (Vorbis for music, PCM for short SFX).

3.4 Core Unity Concepts

GameObjects & Components

Everything in a Unity Scene is a GameObject. A GameObject by itself does nothing — it only gains behaviour through attached Components. This is the Entity-Component pattern.

C# — Adding a Component via Script

```
// Add a Rigidbody2D to this GameObject at runtime
Rigidbody2D rb = gameObject.AddComponent<Rigidbody2D>();
rb.gravityScale = 0f; // disable gravity for top-down shooter
rb.constraints = RigidbodyConstraints2D.FreezeRotation;
```

Prefabs

A Prefab is a saved template of a configured GameObject (with all its components and child objects). Instantiating a Prefab creates a new copy at runtime — essential for bullets, enemies, pickups, and explosions.

C# — Instantiating a Prefab

```
public GameObject bulletPrefab; // drag prefab in Inspector
public Transform firePoint;     // where bullets spawn

void Shoot() {
    // Instantiate(prefab, position, rotation)
    Instantiate(bulletPrefab, firePoint.position, firePoint.rotation);
}
```

Transforms

Every GameObject has a Transform component that stores its position, rotation, and scale. Moving an object means modifying its Transform.

C# — Moving with Transform

```
void Update() {
    float h = Input.GetAxis("Horizontal"); // -1 to 1
    float v = Input.GetAxis("Vertical");   // -1 to 1
    // Time.deltaTime makes speed frame-rate independent
    transform.Translate(new Vector2(h, v) * speed * Time.deltaTime);
}
```

Materials & Lights (Solar System Project)

- Material: Defines how a surface looks — colour, texture, shininess. In 2D, use Sprite Default material or URP/2D Lit materials.
- Directional Light: Simulates the Sun — parallel rays, affects the whole scene.
- Point Light: Emits light in all directions from a point (e.g., planet glow effect).
- Behaviours: Scripts that make planets orbit. Use transform.RotateAround() for circular orbits.

WEEK 4 — Level Design & Game Systems

4.1 Principles of Level Design

Level design is the craft of creating the spaces, challenges, and pacing that players navigate. A well-designed level teaches mechanics implicitly (without a tutorial screen) and escalates challenge gradually.

Core Principles

Principle	Description	Example in 2D Shooter
Safe Introduction	Introduce a mechanic in a low-risk environment	First enemy is slow, isolated
Escalation	Gradually increase difficulty	More enemies, tighter spaces
Contrast	Vary pacing — combat then exploration	Open area after narrow corridor
Reward Placement	Put rewards where players explore	Health pack in side-path
Clear Goals	Player always knows what to do next	Arrow markers, colour cues
Playtesting	Iterate based on real player behaviour	Watch where players die often

4.2 Game System Architecture

A game system is a self-contained set of rules that govern a specific aspect of gameplay. In a 2D shooter, you will implement several interacting systems.

Core Systems in the 2D Shooter Project

Player System

Handles movement, input, health, and shooting. Central to all gameplay.

Enemy System

Spawning, AI movement (patrol/chase), health, death, and loot drops.

Bullet System

Projectile physics, collision detection, damage application, pooling.

Score System

Tracks kills, time, and pickups. Persists between scenes (static or SO).

UI System

HUD (health bar, score, ammo), menus, game-over / win screens.

Audio System

Background music, SFX triggers, volume control via AudioManager.

Level System

Scene loading, level transitions, checkpoint saving.

WEEK 5 — Player Setup: Visual, Controls & Health

5.1 The New Unity Input System

Unity's legacy Input class (Input.GetAxis, Input.GetKey) is being replaced by the New Input System package. The new system is action-based, supports rebinding, and works across multiple devices (keyboard, gamepad, touch) seamlessly.

Setup Steps

- Install via Window → Package Manager → Input System.
- When prompted, enable the new system (Unity restarts).
- Create an Input Actions asset (right-click in Project → Create → Input Actions).
- Define Action Maps (e.g., 'Player') and Actions (e.g., 'Move', 'Fire').
- Generate a C# class from the asset for strongly-typed code.

C# — New Input System Player Movement

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMovement : MonoBehaviour {
    [SerializeField] float moveSpeed = 5f;
    Rigidbody2D rb;
    Vector2 moveInput;

    void Awake() {
        rb = GetComponent<Rigidbody2D>();
    }

    // Called by Input System via PlayerInput component
    void OnMove(InputValue value) {
        moveInput = value.Get<Vector2>(); // normalized direction
    }

    void FixedUpdate() {
        // Physics-based movement – use FixedUpdate, not Update
        rb.linearVelocity = moveInput * moveSpeed;
    }
}
```

■ **EXAM NOTE: Exam Trap:** Movement that uses transform.Translate in Update() causes frame-rate-dependent speed. Always use Rigidbody + FixedUpdate OR multiply by Time.deltaTime.

5.2 Player Health & Damage System

The health system tracks the player's current and maximum HP, processes incoming damage, triggers visual feedback (flash, animation), and handles death.

C# — Complete Health System

```
using UnityEngine;
using UnityEngine.Events;

public class Health : MonoBehaviour {
    [Header('Health Settings')]
    [SerializeField] int maxHealth = 100;
    int currentHealth;
```

```
[Header('Events'')]
public UnityEvent<int> OnHealthChanged; // broadcast to UI
public UnityEvent OnDeath;

void Start() {
    currentHealth = maxHealth;
}

// Called by enemies, hazards, projectiles
public void TakeDamage(int amount) {
    if (amount <= 0) return; // guard clause
    currentHealth -= amount;
    currentHealth = Mathf.Clamp(currentHealth, 0, maxHealth);
    OnHealthChanged?.Invoke(currentHealth); // update HUD
    if (currentHealth <= 0) Die();
}

public void Heal(int amount) {
    currentHealth += amount;
    currentHealth = Mathf.Clamp(currentHealth, 0, maxHealth);
    OnHealthChanged?.Invoke(currentHealth);
}

void Die() {
    OnDeath?.Invoke(); // trigger death animation, game-over
    Destroy(gameObject);
}
}
```

5.3 Player Combat — Shooting

C# — Shooting with Pooling Concept

[illegible]

```
void Update() {  
    // Move forward in local space  
    transform.Translate(Vector2.right * speed * Time.deltaTime);  
}  
  
void OnTriggerEnter2D(Collider2D hit) {  
    Health target = hit.GetComponent<Health>();  
    if (target != null) target.TakeDamage(damage);  
    Destroy(gameObject);          // destroy bullet on impact  
}  
}
```

WEEK 6 — Cameras, Level Environment & Game Loop

6.1 Camera Framing Techniques

Technique	Description	When to Use
Static Camera	Fixed position, no movement	Single-screen arenas
Follow Camera	Camera tracks player position	Scrolling platformers/shooters
Cinemachine	Unity's smart camera package — dampening, confines	Most Unity 2D games
Screen Shake	Trauma-based random offset on impact	Explosions, heavy hits
Parallax Scrolling	Background layers move at different speeds	2D side-scrollers

C# — Simple Follow Camera (without Cinemachine)

```
public class CameraFollow : MonoBehaviour {
    [SerializeField] Transform target; // Player transform
    [SerializeField] float smoothSpeed = 5f;
    [SerializeField] Vector3 offset = new Vector3(0, 0, -10f);

    // LateUpdate ensures player has moved first
    void LateUpdate() {
        Vector3 desiredPos = target.position + offset;
        // Smooth interpolation – avoids jarring snaps
        transform.position = Vector3.Lerp(
            transform.position, desiredPos, smoothSpeed * Time.deltaTime);
    }
}
```

6.2 Game Loop — Management & UI

The Game Manager is a singleton that orchestrates the overall game state — tracking whether the game is playing, paused, or over, and triggering UI changes accordingly.

C# — Singleton Game Manager

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour {
    // Singleton pattern – one instance accessible anywhere
    public static GameManager Instance { get; private set; }

    public int score = 0;
    bool isGameOver = false;

    void Awake() {
        if (Instance != null) { Destroy(gameObject); return; }
        Instance = this;
        DontDestroyOnLoad(gameObject); // persist across scenes
    }

    public void AddScore(int points) {
        score += points;
    }
}
```

```
        UIManager.Instance.UpdateScore(score);
    }

    public void GameOver() {
        if (isGameOver) return;
        isGameOver = true;
        Time.timeScale = 0f;           // pause the game
        UIManager.Instance.ShowGameOver();
    }

    public void RestartGame() {
        Time.timeScale = 1f;           // unpause
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
    }
}
```

6.3 Audio Integration

Unity's audio system uses AudioSource components to play clips and AudioManager for volume control and mixing.

Component	Role	Key Settings
AudioClip	The actual sound file (.wav/.ogg)	—
AudioSource	Component that plays a clip	Loop, Volume, Spatial Blend, Play On Awake
AudioListener	'Ear' of the scene (on Camera)	Only one per scene
AudioMixer	Groups, effects, volume routing	Expose parameters to script

```
C# — Playing Sound Effects

public class SoundManager : MonoBehaviour {
    public static SoundManager Instance;
    [SerializeField] AudioSource sfxSource;
    [SerializeField] AudioSource musicSource;

    void Awake() {
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    public void PlaySFX(AudioClip clip) {
        sfxSource.PlayOneShot(clip); // PlayOneShot allows overlapping sounds
    }

    public void SetMusicVolume(float vol) { // vol: 0.0 - 1.0
        musicSource.volume = vol;
    }
}
```

7.1 Object-Oriented Thinking in Unity

OOP Pillar	Definition	Unity Example
Encapsulation	Hide internal state; expose via methods	private int health; public void TakeDamage()
Inheritance	Child class inherits parent behaviour	Enemy : MonoBehaviour
Polymorphism	Different classes respond differently	enemy.Attack() — different per enemy type
Abstraction	Expose only what is necessary	Interface IDamageable with TakeDamage()

C# — Annotated Script Skeleton

■ **EXAM NOTE:** Lifecycle order for Exam: Awake → OnEnable → Start → FixedUpdate → Update → LateUpdate → OnDisable → OnDestroy

Page 16

[illegible]

■ TIP: Interfaces like `IDamageable` are preferred over inheritance for cross-type damage handling — a bullet should damage enemies AND destructible walls without them sharing a class.

WEEK 8 — Tilemaps, Animation & Advanced Setup

8.1 Tilemaps in Unity

Tilemaps are Unity's built-in system for building 2D grid-based levels efficiently. Instead of placing individual sprite GameObjects, you paint tiles onto a Grid — dramatically reducing draw calls and memory usage.

Tilemap Workflow

- Add a Grid GameObject to the scene (GameObject → 2D Object → Tilemap → Rectangular).
- Open the Tile Palette (Window → 2D → Tile Palette).
- Create a new palette and drag your sprite sheet into it — Unity auto-slices tiles.
- Select a tile from the palette and paint onto the Tilemap in Scene view.
- Add a Tilemap Collider 2D and Composite Collider 2D for physics.

Tile Types Comparison

Tile Type	Description	Best For
Static Tile	Fixed sprite, no logic	Floor, wall, background
Rule Tile	Auto-selects sprite based on neighbours	Auto-tiling walls/terrain
Animated Tile	Cycles through sprite frames	Water, lava, coins
Random Tile	Picks a random sprite from a set	Varied ground textures

8.2 Rule Tiles

A Rule Tile automatically selects the correct sprite based on which neighbouring cells are occupied. This eliminates manual tile placement errors and makes level editing fast.

Example: A Rule Tile for a wall will automatically use a corner sprite when surrounded on two perpendicular sides, a straight sprite when adjacent on two opposite sides, and a T-junction sprite when three sides are filled.

8.3 Animated Tiles

- Create an Animated Tile asset (right-click Project → Create → 2D → Animated Tile).
- Assign the sprite frames in order in the Inspector.
- Set the Minimum Speed and Maximum Speed for frame cycling.
- Paint the tile onto the Tilemap — it animates automatically at runtime.

8.4 Player Physics Refinement

C# — Physics-Based Player with Collision Detection

```
public class PlayerPhysics : MonoBehaviour {
    Rigidbody2D rb;
    [SerializeField] float moveSpeed = 5f;
    [SerializeField] LayerMask groundLayer;
    [SerializeField] Transform groundCheck;
    bool isGrounded;

    void Start() { rb = GetComponent<Rigidbody2D>(); }

    void FixedUpdate() {
```

```

// Circle cast to check if grounded
isGrounded = Physics2D.OverlapCircle(
    groundCheck.position, 0.1f, groundLayer);

float h = Input.GetAxisRaw("Horizontal");
rb.linearVelocity = new Vector2(h * moveSpeed, rb.linearVelocity.y);
}

// Collision callback – called when entering a Tilemap collider
void OnCollisionEnter2D(Collision2D col) {
    if (col.gameObject.CompareTag("Enemy")) {
        GetComponent<Health>().TakeDamage(10);
    }
}
}
}

```

8.5 Canvases & Unity UI

Unity UI is built on Canvas components. Every UI element (Text, Image, Button, Slider) must be a child of a Canvas to render on screen.

Canvas Render Mode	Description	Use Case
Screen Space – Overlay	Renders on top of everything, pixel-perfect	HUD elements, score, health bar
Screen Space – Camera	Rendered by a specific camera	UI with depth effects
World Space	UI exists in 3D world space	Health bars above enemies

C# — Updating a UI Health Bar

```

using UnityEngine;
using UnityEngine.UI;

public class HealthBar : MonoBehaviour {
    [SerializeField] Slider slider; // drag Slider UI component here
    [SerializeField] Health playerHealth; // reference to Health script

    void Start() {
        slider.maxValue = playerHealth.maxHealth;
        slider.value = playerHealth.currentHealth;
        // Subscribe to health changed event
        playerHealth.OnHealthChanged.AddListener(UpdateBar);
    }

    // Called automatically when health changes
    void UpdateBar(int newHealth) {
        slider.value = newHealth;
    }
}

```

SECTION 9 — Interview & Exam Questions

Beginner Level

Q: What is a Game Loop and what three operations does it perform?

A: The game loop runs continuously while the game is active. It processes player input, updates the game state (physics, AI, logic), and renders the current frame. In Unity, `Update()` handles input and logic each frame.

Q: What is the difference between a GameObject and a Component in Unity?

A: A `GameObject` is a container in the Unity scene. By itself it does nothing — it gains functionality through Components (scripts, colliders, renderers, audio sources, etc.) attached to it.

Q: What is a Prefab and why do we use them?

A: A Prefab is a saved template of a configured `GameObject`. We use Prefabs to instantiate multiple copies of the same object at runtime (bullets, enemies, particles) without recreating them manually each time.

Q: What does `Time.deltaTime` do?

A: It provides the time elapsed since the last frame in seconds. Multiplying movement by `Time.deltaTime` makes movement frame-rate independent — the object moves the same real-world distance per second regardless of frame rate.

Q: Name three game development team roles and their responsibilities.

A: Game Designer (GDD, mechanics), Programmer (code, systems), Artist (visual assets, animations). Others include QA Tester, Audio Engineer, and Producer.

Intermediate Level

Q: Explain the difference between `Update()`, `FixedUpdate()`, and `LateUpdate()`.

A: `Update()` runs every frame — use for input and game logic. `FixedUpdate()` runs at a fixed physics timestep (default 0.02s) — use for Rigidbody physics to ensure consistent simulation. `LateUpdate()` runs after all `Update()` calls — ideal for camera follow so the player has already moved.

Q: What is the Singleton pattern and how is it used for a Game Manager?

A: A Singleton ensures only one instance of a class exists and provides global access. In Unity, set a static `Instance` property in `Awake()`, and call `Destroy(gameObject)` if another instance exists. Use `DontDestroyOnLoad()` to persist across scene loads.

Q: What is the difference between `OnCollisionEnter2D` and `OnTriggerEnter2D`?

A: `OnCollisionEnter2D` fires when two solid colliders physically collide with resolution (objects bounce/stop). `OnTriggerEnter2D` fires when one collider (marked as `Is Trigger`) overlaps another without physical response — used for pickups, damage zones, checkpoints.

Q: What is a Rule Tile and when would you use it over a Static Tile?

A: A Rule Tile automatically selects the correct sprite based on neighbouring cells. Use it for terrain or walls where you want auto-tiling — it eliminates manual sprite selection errors and speeds up level design significantly.

Q: How does the New Input System differ from the legacy Input class?

A: The New Input System is action-based and device-agnostic — you define 'Move' and 'Fire' actions once and they work with keyboard, gamepad, and touch automatically. It supports runtime rebinding, input buffering, and player-specific input. The legacy Input class is simpler but lacks these features.

Advanced Level

Q: Explain Object Pooling and why it is preferable to Instantiate/Destroy for bullets.

A: Object Pooling pre-creates a set of objects and recycles them instead of creating/destroying them each time. `Instantiate()` and `Destroy()` trigger garbage collection, causing frame-rate spikes. A pool deactivates objects when 'destroyed' and reactivates them when 'spawned' — eliminating GC pressure entirely.

Q: How would you implement a scalable enemy AI system in Unity?

A: Use a State Machine (Idle → Patrol → Chase → Attack → Dead) driven by a base Enemy class with virtual methods. Each state is a separate class implementing IState. A Finite State Machine (FSM) manager switches states based on triggers (player distance, health threshold). For complex behaviours, use Unity's NavMesh for pathfinding.

Q: What is a ScriptableObject and when should you use it?

A: A ScriptableObject is a data container asset that exists outside of scenes — it is not attached to GameObjects. Use it for game data (enemy stats, item definitions, audio configs) shared across multiple prefabs to avoid data duplication and enable designer-friendly tuning without code changes.

Scenario-Based Questions

Scenario: Your 2D shooter runs at 60 FPS on PC but only 30 FPS on the target device. How do you diagnose and fix performance issues?

Solution: Use Unity Profiler (Window → Analysis → Profiler) to identify bottlenecks. Common fixes: reduce Draw Calls with sprite atlases, implement Object Pooling for bullets, use Composite Collider for Tilemaps, reduce audio channels, set camera Culling Mask to exclude unnecessary layers, and use LOD/culling for off-screen objects.

Scenario: Your player clips through thin walls when moving fast. What causes this and how do you fix it?

Solution: This is Tunnelling — the physics step moves the object so far in one FixedUpdate that it passes through thin geometry. Fix: set Rigidbody2D Collision Detection Mode to Continuous instead of Discrete. Also ensure walls are at least as thick as the maximum distance the player travels per physics step.

SECTION 10 — Quick-Revision Summary

Key Concepts Cheat Sheet

Topic	Must-Know Fact
Game Loop	Input → Update → Render. In Unity: Update() every frame
deltaTime	Elapsed time since last frame; multiply for frame-rate independence
Awake vs Start	Awake: always called first. Start: called before first frame if enabled
FixedUpdate	Fixed 0.02s timestep — use for physics (Rigidbody)
LateUpdate	After all Updates — use for camera follow
Prefab	Saved reusable GameObject template — Instantiate() creates a copy
Singleton	static Instance; DontDestroyOnLoad for cross-scene persistence
OnTrigger vs Collision	Trigger = overlap, no physics. Collision = physical response
Rule Tile	Auto-selects sprite based on neighbouring tiles
New Input System	Action-based, device-agnostic, supports rebinding
Canvas Modes	Overlay, Camera, World Space — know when to use each
ScriptableObject	Data container asset — not a MonoBehaviour, not scene-dependent
Game Phases	Pre-Production → Production → Alpha → Beta → Gold
Iron Triangle	Fast, Cheap, Good — pick two
Unity Lifecycle Order	Awake→OnEnable→Start→FixedUpdate→Update→LateUpdate→OnDestroy

Top 10 Exam Tips

- 1. Always explain WHY a concept matters, not just what it is — examiners award marks for depth.
- 2. Time.deltaTime is mentioned in almost every movement question — never forget it.
- 3. Draw a diagram of the Game Loop if asked — Input → Update → Render is the correct order.
- 4. The Unity lifecycle order (Awake → Start → Update → ...) is a guaranteed exam topic.
- 5. Know the difference between Trigger and Collision callbacks — a one-mark question every year.
- 6. Singleton pattern: static Instance + DontDestroyOnLoad — memorise this implementation.
- 7. Prefabs exist to enable reuse and Instantiate — link this to the DRY (Don't Repeat Yourself) principle.
- 8. For Tilemap questions: Palette → Rule Tiles → Composite Collider is the complete workflow.
- 9. Alpha vs Beta: Alpha = feature-complete; Beta = content-complete. Do not confuse these.
- 10. Always mention FixedUpdate for physics questions — Update() + physics = wrong answer.

References & Further Reading:

- Unity Official Documentation — <https://docs.unity3d.com>
- Unity Learn (Free Tutorials) — <https://learn.unity.com>
- C# Programming Guide — Microsoft Docs (<https://learn.microsoft.com/en-us/dotnet/csharp/>)

- Book: 'Game Programming Patterns' — Robert Nystrom (free online at gameprogrammingpatterns.com)
- Book: 'Unity in Action' — Joseph Hocking (Manning Publications)
- YouTube: Code Monkey, Brackeys (archived), Game Maker's Toolkit — top channels for Unity learners
- Unity Input System Docs — <https://docs.unity3d.com/Packages/com.unity.inputsystem@latest>

End of Notes — Good Luck on Your Exam!